Using Autograd in PyTorch to Solve a Regression Problem

by **Adrian Tam** on April 8, 2023 in **Deep Learning with PyTorch** 0

We usually use PyTorch to build a neural network. However, PyTorch can do more than this. Because PyTorch is also a tensor library with automatic differentiation capability, you can easily use it to solve a numerical optimization problem with gradient descent. In this post, you will learn how PyTorch's automatic differentiation engine, autograd, works.

After finishing this tutorial, you will learn:

- What is autograd in PyTorch
- How to make use of autograd and an optimizer to solve an optimization problem

Kick-start your project with my book Deep Learning with PyTorch. It provides **self-study tutorials** with **working code**.

Let's get started.

Using autograd in PyTorch to solve a regression problem.
Photo by Billy Kwok. Some rights reserved.

Overview

This tutorial is in three parts; they are:

- Autograd in PyTorch

- Using Autograd for Polynomial Regression
- Using Autograd to Solve a Math Puzzle

Autograd in PyTorch

In PyTorch, you can create tensors as variables or constants and build an expression with them. The expression is essentially a function of the variable tensors. Therefore, you may derive its derivative function, i.e., the differentiation or the gradient. This is the foundation of the training loop in a deep learning model. PyTorch comes with this feature at its core.

It is easier to explain autograd with an example. In PyTorch, you can create a constant matrix as follows:

```
import torch

x = torch.tensor([1, 2, 3])
print(x)
print(x.shape)
print(x.dtype)
```

The above prints:

```
tensor([1, 2, 3])
torch.Size([3])
torch.int64
```

This creates an integer vector (in the form of a PyTorch tensor). This vector can work like a NumPy vector in most cases. For example, you can do `x+x` or `2*x`, and the result is just what you would expect. PyTorch comes with many functions for array manipulation that match NumPy, such as `torch.transpose` or `torch.concatenate`.

But this tensor is not assumed to be a variable for a function in the sense that differentiation with it is not supported. You can create tensors that work like a variable with an extra option:

```
import torch

x = torch.tensor([1., 2., 3.], requires_grad=True)
print(x)
print(x.shape)
print(x.dtype)
```

This will print:

```
tensor([1., 2., 3.], requires_grad=True)
torch.Size([3])
torch.float32
```

Note that, in the above, a tensor of floating point values was created. It is required because differentiation requires floating points, not integers.

The operations (such as `x+x` and `2*x`) can still be applied, but in this case, the tensor will remember how it got its values. You can demonstrate this feature in the following:

```
import torch

x = torch.tensor(3.6, requires_grad=True)
y = x * x
y.backward()
print(x.grad)
```

This prints:

```
tensor(7.2000)
```

What it does is the following: This defined a variable `x` (with value 3.6) and then computed `y=x*x` or $y = x^2$. Then you ask for the differentiation of y. Since y obtained its value from x, you can find the derivative $\frac{dy}{dx}$ at `x.grad`, in the form of a tensor, immediately after you run `y.backward()`. You know $y = x^2$ means $y' = 2x$. Hence the output would give you a value of $3.6 \times 2 = 7.2$.

Using Autograd for Polynomial Regression

How is this feature in PyTorch helpful? Let's consider a case where you have a polynomial in the form of $y = f(x)$, and you are given several (x, y) samples. How can you recover the polynomial $f(x)$? One way is to assume a random coefficient for the polynomial and feed in the samples (x, y). If the polynomial is found, you should see the value of y matches $f(x)$. The closer they are, the closer your estimate is to the correct polynomial.

This is indeed a numerical optimization problem where you want to minimize the difference between y and $f(x)$. You can use gradient descent to solve it.

Let's consider an example. You can build a polynomial $f(x) = x^2 + 2x + 3$ in NumPy as follows:

```
import numpy as np

polynomial = np.poly1d([1, 2, 3])
print(polynomial)
```

This prints:

```
   2
1 x + 2 x + 3
```

You may use the polynomial as a function, such as:

```
print(polynomial(1.5))
```

And this prints `8.25`, for $(1.5)^2 + 2 \times (1.5) + 3 = 8.25$.

Now you can generate a number of samples from this function using NumPy:

```
N = 20   # number of samples

# Generate random samples roughly between -10 to +10
X = np.random.randn(N,1) * 5
Y = polynomial(X)
```

In the above, both `X` and `Y` are NumPy arrays of the shape `(20,1)`, and they are related as $y = f(x)$ for the polynomial $f(x)$.

Now, assume you do not know what the polynomial is except it is quadratic. And you want to recover the coefficients. Since a quadratic polynomial is in the form of $Ax^2 + Bx + C$, you have three unknowns to find. You can find them using the gradient descent algorithm you implement or an existing gradient descent optimizer. The following demonstrates how it works:

```
import torch

# Assume samples X and Y are prepared elsewhere

XX = np.hstack([X*X, X, np.ones_like(X)])

w = torch.randn(3, 1, requires_grad=True)  # the 3 coefficients
x = torch.tensor(XX, dtype=torch.float32)  # input sample
y = torch.tensor(Y, dtype=torch.float32)   # output sample
optimizer = torch.optim.NAdam([w], lr=0.01)
print(w)

for _ in range(1000):
    optimizer.zero_grad()
    y_pred = x @ w
    mse = torch.mean(torch.square(y - y_pred))
    mse.backward()
    optimizer.step()

print(w)
```

The `print` statement before the for loop gives three random numbers, such as:

```
tensor([[1.3827],
        [0.8629],
        [0.2357]], requires_grad=True)
```

But the one after the for loop gives you the coefficients very close to that in the polynomial:

```
tensor([[1.0004],
        [1.9924],
        [2.9159]], requires_grad=True)
```

What the above code does is the following: First, it creates a variable vector `w` of 3 values, namely the coefficients A, B, C. Then you create an array of shape $(N, 3)$, in which N is the number of samples in the array `X`. This array has 3 columns: the values of x^2, x, and 1, respectively. Such an array is built from the vector `X` using the `np.hstack()` function. Similarly, you build the TensorFlow constant `y` from the NumPy array `Y`.

Afterward, you use a for loop to run the gradient descent in 1,000 iterations. In each iteration, you compute $x \times w$ in matrix form to find $Ax^2 + Bx + C$ and assign it to the variable `y_pred`. Then, compare `y` and `y_pred` and find the mean square error. Next, derive the gradient, i.e., the rate of change of the mean square error with respect to the coefficients `w` using the `backward()` function. And based on this gradient, you use gradient descent to update `w` via the optimizer.

In essence, the above code will find the coefficients `w` that minimizes the mean square error.

Putting everything together, the following is the complete code:

```
import numpy as np
import torch

polynomial = np.poly1d([1, 2, 3])
N = 20   # number of samples

# Generate random samples roughly between -10 to +10
X = np.random.randn(N,1) * 5
Y = polynomial(X)

# Prepare input as an array of shape (N,3)
XX = np.hstack([X*X, X, np.ones_like(X)])

# Prepare tensors
w = torch.randn(3, 1, requires_grad=True)  # the 3 coefficients
x = torch.tensor(XX, dtype=torch.float32)  # input sample
y = torch.tensor(Y, dtype=torch.float32)   # output sample
optimizer = torch.optim.NAdam([w], lr=0.01)
print(w)

# Run optimizer
for _ in range(1000):
    optimizer.zero_grad()
    y_pred = x @ w
    mse = torch.mean(torch.square(y - y_pred))
    mse.backward()
    optimizer.step()

print(w)
```

Using Autograd to Solve a Math Puzzle

In the above, 20 samples were used, which is more than enough to fit a quadratic equation. You may use gradient descent to solve some math puzzles as well. For example, the following problem:

```
[ A ]  +  [ B ]  =  9
  +         -
[ C ]  -  [ D ]  =  1
  =         =
  8         2
```

In other words, to find the values of A, B, C, D such that:

$$
\begin{aligned}
A + B &= 9 \\
C - D &= 1 \\
A + C &= 8 \\
B - D &= 2
\end{aligned}
$$

This can also be solved using autograd, as follows:

```
import random
import torch

A = torch.tensor(random.random(), requires_grad=True)
B = torch.tensor(random.random(), requires_grad=True)
C = torch.tensor(random.random(), requires_grad=True)
D = torch.tensor(random.random(), requires_grad=True)

# Gradient descent loop
EPOCHS = 2000
optimizer = torch.optim.NAdam([A, B, C, D], lr=0.01)
for _ in range(EPOCHS):
    y1 = A + B - 9
    y2 = C - D - 1
    y3 = A + C - 8
    y4 = B - D - 2
    sqerr = y1*y1 + y2*y2 + y3*y3 + y4*y4
    optimizer.zero_grad()
    sqerr.backward()
    optimizer.step()

print(A)
print(B)
print(C)
print(D)
```

There can be multiple solutions to this problem. One solution is the following:

```
tensor(4.7191, requires_grad=True)
tensor(4.2808, requires_grad=True)
tensor(3.2808, requires_grad=True)
tensor(2.2808, requires_grad=True)
```

Which means $A = 4.72$, $B = 4.28$, $C = 3.28$, and $D = 2.28$. You can verify this solution fits the problem.

The above code defines the four unknowns as variables with a random initial value. Then you compute the result of the four equations and compare it to the expected answer. You then sum up the squared error and ask PyTorch's optimizer to minimize it. The minimum possible square error is zero, attained when our solution exactly fits the problem.

Note the way PyTorch produces the gradient: You ask for the gradient of `sqerr`, which it noticed that, among other things, only `A`, `B`, `C`, and `D` are its dependencies that `requires_grad=True`. Hence four gradients are found. You then apply each gradient to the respective variables in each iteration via the optimizer.

Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Articles:

- Autograd mechanics
- Automatic differentiation package – torch.autograd

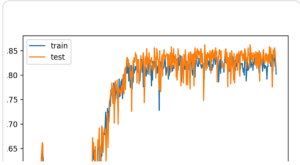
Summary

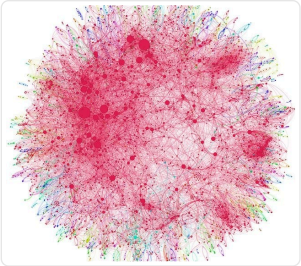
In this post, we demonstrated how PyTorch's automatic differentiation works. This is the building block for carrying out deep learning training. Specifically, you learned:

- What is automatic differentiation in PyTorch
- How you can use gradient tape to carry out automatic differentiation
- How you can use automatic differentiation to solve an optimization problem

More On This Topic






Understand Your Problem and Get Better Results Using…


How to Define Your Machine Learning Problem

About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.
View all posts by Adrian Tam →

‹ Manipulating Tensors in PyTorch

Building Multilayer Perceptron Models in PyTorch ›

No comments yet.

Leave a Reply

Name (required)

Email (will not be published) (required)

SUBMIT COMMENT

Welcome!
I'm *Jason Brownlee* PhD
and I **help developers** get results with **machine learning**.
Read more

Never miss a tutorial: